

Week 2

Prerit Advani

June 2026

1 Solving MDPs through Model-Free Methods (Reinforcement Learning)

When the environment is perfectly modeled, we possess exact knowledge of the Transition Probability $P(s'|s, a)$ and the Reward Function $R(s, a, s')$. In this theoretical setting, we can use **Dynamic Programming (DP)** to solve for the State-Value Function (V) or Action-Value Function (Q) functions. By iteratively applying the Bellman equations, we can compute the Policy (π) that yields the highest possible return without ever needing to interact with the environment.

But, In most real-world scenarios, the transition probabilities and reward functions are unknown. Here, we use **Reinforcement Learning (RL)** to estimate these values through trial and error.

1.1 temporal Differencing (TD)

Methods that learn by observing the differences between predicted rewards and actual rewards received after taking an action. There are primarily 2 temporal differencing methods used:

1.1.1 Q-Learning (Off-Policy)

The agent learns the value of the optimal policy regardless of the actions it is actually taking. It updates its $Q(s, a)$ estimates based on the maximum possible future reward.

1.1.2 SARSA (On-policy)

The agent learns the value of the policy it is currently following, including the exploratory moves it makes. It updates $Q(s, a)$ based on the action it actually takes next.

1.2 Monte Carlo Methods

These learn from complete episodes of experience by averaging the total returns received after visiting a state. Unlike TD learning, they only update after an episode ends.

1.3 Deep Actor-Critic Methods

While traditional TD methods (like Q-Learning) learn value functions to derive a policy, modern Deep RL often employs **Actor-Critic** architectures. These methods combine the benefits of policy-based learning (learning the strategy directly) and value-based learning (using a *critic* to evaluate actions via TD errors). Examples include:

- **PPO (Proximal Policy Optimization):** An on-policy method focused on stable, incremental policy updates.
- **DDPG (Deep Deterministic Policy Gradient):** An off-policy method designed for continuous action spaces.
- **SAC (Soft Actor-Critic):** An off-policy method that maximizes both reward and entropy for robust exploration.

2 Bellman Expectation and Optimality Equations:

By the terms defined in the Previous week, we have:

$$v(S) = E[G_t | S_t = s]$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

The value function can be decomposed into two parts using the linearity of expectation:

$$v(s) = E[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) | S_t = s]$$

$$\implies v(s) = E[R_{t+1} + \gamma G_{t+1} | S_t = s] = E[R_{t+1} + \gamma v(S_{t+1}) | S_t = s]$$

$$\implies v(s) = E[R_{t+1}] + \gamma E[v(S_{t+1}) | S_t = s] = \mathcal{R}_f + \sum_{s' \in S} P_{ss'} v(s')$$

Matrix Form:

$$V = R + \gamma PV$$

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & \ddots & \vdots \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

- **v**: The value function vector, where each element $v(i)$ represents the value of state i .
- **r**: The immediate reward vector, where each $\mathcal{R}_i$ is the expected reward received in state i .
- γ : The discount factor, which determines the present value of future rewards.
- **P**: The transition probability matrix, where the element $\mathcal{P}_{ij}$ represents the probability of transitioning from state i to state j .

This is a linear equation and can be solved as follows:

$$(I - \gamma P)V = R \implies V = (I - \gamma P)^{-1}R$$

The computational complexity is $O(n^3)$. Hence, feasible only for small MRPs. The larger ones are solved using iterative methods like:

- Dynamic Programming
- Monte Carlo evaluation
- Temporal Differencing

When we have decide actions at each step using some policy, the process is known as Markov Decision Process.

We need to choose the action that maximizes reward at each step and this maximum value is known as the optimal value function of the state. $q_\pi(s, a)$ Denotes the reward starting at state $S_t = s$ with the action $A_t = a$ following the policy π after the first action.

$$\begin{aligned} q_\pi(s, a) &= E_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \\ &= \mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} v_\pi(s') \\ v_\pi(s) &= \sum_{a \in A} \pi(a|s) q_\pi(s, a) \\ &= \sum_{a \in A} \pi(a|s) (\mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} v_\pi(s')) \\ q_\pi(s, a) &= \mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} (\sum_{a \in A} \pi(a|s') q_\pi(s', a)) \end{aligned}$$

Matrix Form:

$$\begin{aligned} V_\pi &= \mathcal{R}_\pi + \gamma P_\pi V_\pi \\ \implies V_\pi &= (I - \gamma P_\pi)^{-1} \mathcal{R}_\pi \end{aligned}$$

Optimal Value Function:

$$\begin{aligned} v_*(s) &= \max_\pi v_\pi(s) \\ q_*(s, a) &= \max_\pi q_\pi(s, a) \end{aligned}$$

Partial Ordering over Policies:

A policy π is considered better than or equal to another policy π' (denoted as $\pi \geq \pi'$) if the state-value function of π is greater than or equal to that of π' for all states s :

$$\pi \geq \pi' \text{ if } v_\pi(s) \geq v_{\pi'}(s), \forall s$$

Theorem of Optimal Policy:

For any Markov Decision Process (MDP):

- Existence: There exists an optimal policy π_* that is better than or equal to all other policies ($\pi_* \geq \pi, \forall \pi$).
- Optimal Value Function: All optimal policies achieve the optimal state-value function, denoted as:

$$v_{\pi_*}(s) = v_*(s)$$

- Optimal Action-Value Function: All optimal policies achieve the optimal action-value function, denoted as:

$$q_{\pi_*}(s, a) = q_*(s, a)$$

Bellman Optimality Equations:

$$q_*(s, a) = \mathcal{R}_s^a + \sum_{s' \in S} P_{ss'} v_*(s')$$

$$v_*(s) = \max_a (q_*(s, a))$$

bellman Optimality equation is non linear and has no closed form in general.
There are various iterative algorithms such as:

- Value Iteration
- Policy Iteration
- Q learning
- Sarsa

Glossary

MDP Markov Decision Process. A mathematical framework used to model decision-making situations where outcomes are partly random and partly under the control of a decision-maker.

State Space (S) The set of all valid situations, configurations, or positions ($s \in S$) that the agent can occupy within the environment.

Action Space (A) The set of all possible moves, choices, or decisions ($a \in A$) available to the agent in its current situation.

Transition Probability (P) The probability $P(s' | s, a)$ that executing a specific action a in state s will cause the environment to shift to a specific next state s' . This accounts for natural randomness or noise in the environment.

Reward Function (R) The immediate numerical feedback $R(s, a, s')$ given to the agent by the environment right after moving from state s to state s' via action a .

Policy (π) The agent's strategic brain or rulebook. Formally written as $\pi(a | s)$, it maps a given state to the probability distribution of selecting each available action. The ultimate goal of the agent is to find the Optimal Policy (π^*).

State-Value Function (V) Denoted as $V(s)$, it measures how good it is to be in a specific state s by calculating the total expected future reward accumulated from that point onward, assuming the agent follows its current strategy until the end.

Action-Value Function (Q) The Quality function, denoted as $Q(s, a)$, which calculates the total expected long-term return if the agent commits to a specific initial action a from state s , and follows its policy thereafter.

Discount Factor A factor $\gamma \in [0, 1]$ that determines the present value of future rewards.

Transition Probability The probability $P(s' | s, a)$ of transitioning to state s' from state s after taking action a .

Reward Function The function $R(s, a, s')$ that gives the immediate reward for transitioning from state s to s' via action a .